Engineering, Built Environment and IT
Department of Computer Science
Programming Languages
COS 333

Examination
25 June 2024

Examiner: Mr W. S. van Heerden
External Examiner: Mr Willem H. K. Bester (Stellenbosch)
Instructions:
Read the questions carefully and answer all questions.
This section comprises of **28** questions, and a total of **40** marks.
You have **3** hours to complete this test.
This test is an *online* assessment:
The test will automatically submit when the time (3 hours) expires. You can also submit the test yourself if you are done ahead of time.
Be sure to complete the assignment submissions of the two practical implementation questions (Questions 5 and 6). If you do not complete the submission of your implementations, there is no way it can be accessed for assessment. There are unlimited upload opportunities for the implementation assignments, but only the last submission will be assessed. Please make absolutely sure that you upload the correct file for each assignment.
This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.
You are not allowed to consult any sources other than the Practical 2 specification, the MIT/GNU Scheme Reference Manual (provided as documentation for Practical 2), the Practical 3 specification, the SWI-Prolog Reference Manual (provided as documentation for Practical 3), and the Practical 4 specification.
Your Scheme submission for Question 5 must be working Scheme code that will successfully interpret using version 8.8 of the DrRacket interpreter. You may (and should) use this interpreter to test your submission. Once you have finished writing your program, make sure it is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.
Your Prolog submission for Question 6 must be working Prolog code that will successfully interpret using version 9.0.4 of the SWI-Prolog interpreter. You may (and should) use this interpreter to test your submission. Once you have finished writing your program, make sure it is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot.
You may use a non-programmable calculator.
You may not use any other programmable devices.
Integrity Statement:
The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

QUESTION 1

1. There are often trade-offs between the programming language evaluation criteria we have used in this module. Consider the following statements, and **select** the one that most correctly describes a likely tradeoff when readability improves.

- The cost of execution becomes worse.
- The cost of maintenance becomes worse.
- The cost of implementation becomes worse.
- Reliability becomes worse.

1 points

QUESTION 2

1. An additional programming language evaluation criterion that is not focussed on heavily in this course is generality. Consider the following programming languages, and **select** the one that has the best generality.

- Fortran I
- ALGOL 60
- SIMULA 67
- PL/I

1 points

QUESTION 3

1. Consider the following statements about the Pascal and C programming languages, and **select** the one that is the most correct.

- Pascal is more readable than C.
- Pascal is more writable than C.
- Pascal is less reliable than C.
- Pascal has a higher cost of maintenance than C.

1 points

QUESTION 4

1. An additional programming language evaluation criterion that is not focussed on heavily in this course is portability. Consider the following programming languages, and **select** the one that is the most portable.

- Fortran 90
- C++
- PL/I
- JavaScript

1 points

QUESTION 5

1. Write a Scheme function named `removePositiveOddValues`, which receives one parameter. The parameter is a simple list containing only numeric atoms. The function should yield (not print out) a list containing the values in the parameter list, in their original order, with all the positive odd values removed.

   For example, the function application

   ```
   (removePositiveOddValues '(3 9))
   ```

   should yield an empty list because all the numeric atoms in the parameter list are positive odd values. As another example, the function application

   ```
   (removePositiveOddValues '(-5 3 0 5 9))
   ```

   should yield the list `(-5 0)` because `3`, `5`, and `9` are positive odd numbers, and are therefore removed from the list, while `-5` and `0` remain.
   Submit your program code in a file named `s99999999.scm` (where `99999999` is your student number) to the assignment submission slot labelled "Examination: Scheme Function", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

   Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter using the sicp language collections (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may only use the following built-in functions in the way they are used in the slides and textbook (failure to do so will result in a zero mark for this question):

  - Function construction: `lambda`, `define`
  - Binding: `let`
  - Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
  - Boolean values: `#t`, `#f`
  - Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
  - Logical predicates: `and`, `or`, `not`
  - List predicates: `list?`, `null?`
  - Conditionals: `if`, `cond`, `else`
  - Quoting: `quote`, `'`
  - List manipulation: `list`, `car`, `cdr`, `cons`
  - Input and output: `display`, `printf`, `newline`, `read`

- ○ Yes
- ○ No

5 points

QUESTION 6

1. **Write** a Prolog proposition called `doublePositives(L1, L2)`, which succeeds if `L2` is a list containing all the elements in `L1`, in their original order, with every positive value doubled in magnitude, and every non-positive value unchanged. You may assume that lists `L1` and `L2` are simple lists containing only integers. For example, the query:

```
?- doublePositives([-4, -9], X).
```

should be true with the instantiation `X = [-4, -9]`. This is because there are no elements that are positive in the list `[-4, -9]`. Similarly, the query

```
?- doublePositives([-3, 6, 7, -1, -5], X).
```

should be true with the instantiation `X = [-3, 12, 14, -1, -5]`. This is because `-3`, `-1`, and `-5` are not positive and are therefore not doubled, while `6` is doubled to give the value `12` and `7` is doubled to give the value `14`.

Hint 1: You can use comparison operators as terms in Prolog. These operators are similar to what you are used to in imperative languages. The operators < (less than), =< (less than or equal to), > (greater than), and >= (greater than or equal to) are all valid. As a very simple example, you could use a comparison operator as follows:

```
lessThan(X, Y) :- X < Y.
```

The following queries are then possible:

```
?- lessThan(1, 2).
true.
?- lessThan(2, 1).
false.
```

Note that this is just an example, and does not mean you should use this `lessThan` proposition in your implementation (although you may if you wish to).

Hint 2: Multiplication is represented by `*` in Prolog.

Submit your program code in a file named `s99999999.pl` (where `99999999` is your student number) to the assignment submission slot labelled "Examination: Prolog Proposition", and select the "Yes" answer below once you have done so. If you do not make a submission, select the "No" answer below.

Take careful note of the following requirements:

- Your submission must be working Prolog code that will be successfully interpreted by the SWI-Prolog interpreter installed on the Windows computers in the Informatorium. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3. Once you have finished writing your program, save it in a text file with the appropriate name, and submit it to the correct upload slot.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, and built-in predicates discussed in the textbook, slides, and hints**. In particular, do not use if-then, if-then-else, and similar constructs. You may NOT use any more complex predicates provided by the Prolog system itself. In other words, you must write all your own propositions. **Failure to observe this rule will result in all marks for this question being forfeited**.

○ Yes

○ No

5 points

QUESTION 7

1. Binding can occur at different times. Consider the following options, and **select** the one that most correctly describes the binding time of a standard 32-bit signed integer type in a statically typed programming language to either a sign-and-magnitude or two's complement representation.

- Language design time.
- Language implementation time.
- Compile time.
- Load time.
- Runtime.

1 points

QUESTION 8

1. Consider the following program code in C# (line numbers are provided for reference purposes):

```
1. {
2.     int y;
3.     {
4.         int x = 5;
5.     }
6.     int x = 2;
7. }
```

Consider the following statements about the above program code, and **select** the one that is the most correct.

- The provided program code does not compile because the variable `y` is not initialised.
- The provided program code does not compile because the variable `x` on line 4 hides the variable `x` on line 6.
- The provided program code does not compile because the variable `x` on line 6 hides the variable `x` on line 4.
- The provided program code compiles and runs successfully.

1 points

QUESTION 9

1. Consider the following statements about character string types in programming languages, and **select** the one that is the most correct.

- In programming languages that represent character strings using only classes, character strings are always mutable.
- In programming languages that represent character strings using only classes, character strings are always immutable.
- Limited dynamic length strings have a higher overall hardware resource cost than static length strings, and a lower overall hardware resource cost than dynamic length strings.
- C supports character strings as a primitive type.

1 points

QUESTION 10

1. Consider the following statements about unions, and **select** the one that is the most correct.

- Unions are closely related to classes in object-oriented programming, but are more limited in terms of functionality.
- Discriminated unions are more memory efficient than free unions.
- Unions can be used as a mechanism for returning multiple values from a subprogram.
- Unions are used when memory cost is of great concern.

1 points

QUESTION 11

1. Consider the following statements about arithmetic operations in Scheme, and **select** the one that is the most correct.

- The order in which arithmetic operations are evaluated in Scheme is unambiguously determined based on clear precedence levels and associativity rules.
- The order in which arithmetic operations are evaluated in Scheme is unambiguously determined based on function and parameter evaluation.
- The order in which arithmetic operations are evaluated in Scheme is unambiguously determined because all operators have the same precedence level, and associate from left to right.
- The order in which arithmetic operations are evaluated in Scheme is ambiguous because function parameters are evaluated in no particular order.

1 points

QUESTION 12

1. Consider the following statements about expressions in Ada, and **select** the one that is the most correct.

- The lack of coercions in Ada makes the language more readable.
- The lack of coercions in Ada makes the language less reliable.
- The common use of coercions in Ada makes the language more writable.
- The common use of coercions in Ada makes the language less reliable.

1 points

QUESTION 13

1. Consider the following program code in a hypothetical programming language:

```
int count = 5
double result = 0.0

while ((count > 0) and (result = (10.0 / count))) {
   print result
   count = count - 1
}
```

Assume that the programming language uses C-like syntax, implements assignment as an expression, uses `and` to represent a logical and operation, and interprets integers as Boolean values in the same way that C does. Also assume that the program is syntactically correct. Consider the following statements, and **select** the one that is the most correct.

- The provided program code produces an error regardless of whether the logical `and` is short-circuit evaluated or not.
- The provided program code produces an error if the logical `and` is short-circuit evaluated.
- The provided program code produces an error if the logical `and` is not short-circuit evaluated.
- The provided program code produces no error regardless of whether the logical `and` is short-circuit evaluated or not.

1 points

QUESTION 14

1. Consider the following statements about unary assignment operators, and **select** the one that is the most correct.

- ○ Unary assignment operators reduce readability, if they are overloaded to provide prefix and postfix versions.
- ○ Unary assignment operators reduce writability, if they are overloaded to provide prefix and postfix versions.
- ○ Unary assignment operators reduce the execution cost of a programming language.
- ○ Unary assignment operators are essential within a programming language.

1 points

QUESTION 15

1. Consider the following statements, and **select** only those that are true about counter-controlled loops. Note that incorrectly selected options will be penalised.

- ☐ Counter-controlled loops are required in order to implement any flowchart-represented algorithm.
- ☐ Counter-controlled loops in Python are more reliable than counter-controlled loops in C.
- ☐ Counter-controlled loops in C are more readable than counter-controlled loops in Java.
- ☐ Counter-controlled loops are not directly supported in purely functional programming languages.

2 points

QUESTION 16

1. Consider the following statements about disambiguating nested selection statements, and **select** only the ones that are correct. Note that incorrectly selected options will be penalised.

- ☐ Python's approach to disambiguating nested selection statements is less readable than Java's approach.
- ☐ Python's approach to disambiguating nested selection statements is less reliable than Java's approach.
- ☐ Ruby and Perl use very similar approaches to disambiguating nested selection statements, but Perl's approach is more writable than Ruby's approach.
- ☐ Python and Ruby use very similar approaches to disambiguating nested selection statements, but Python's approach is more writable than Ruby's approach.

2 points

QUESTION 17

1. Consider the following program code in a hypothetical programming language that supports guarded commands:

```
var myVal = 2

do (myVal < 0) -> print("A ")
                  myVal = 1
[] (myVal > 0) -> print("B ")
                  myVal = myVal - 1
od
```

Assume that the `print` subprogram prints its string output to the screen on a single line. **Provide the output** of the program in the space provided. If the program produces no output, type "Nothing" without the quotation marks. If the program produces a compile-time error, type "Compilation error" without the quotation marks. If the program produces a runtime error, type "Runtime error", without quotation marks, at the point where the runtime error occurs. Note that there is a single space at the end of each of the outputs produced by the `print` subprograms.

1 points

QUESTION 18

1. Consider the following statements about subprograms that are defined outside a class definition, and **select** the one that is the most correct.

- Ruby subprograms that are defined outside a class definition are more writable than C++ subprograms that are defined outside a class definition.
- Ruby subprograms that are defined outside a class definition are less writable than C++ subprograms that are defined outside a class definition.
- Ruby subprograms that are defined outside a class definition are more orthogonal than C++ subprograms that are defined outside a class definition.
- Ruby subprograms that are defined outside a class definition behave in exactly the same way as C++ subprograms that are defined outside a class definition.

1 points

QUESTION 19

1. Consider the following statements about procedures and functions, and **select** the one that is the most correct.

- A procedure can produce a result if all its parameters use pass-by-value semantics.
- A procedure is guaranteed to produce no side effect if it produces a result.
- A function is guaranteed to produce a side effect if it produces a result.
- A procedure can more easily produce multiple results than a function can.

1 points

QUESTION 20

1. Consider the following program code in a hypothetical programming language:

```
doSomething(int first, int second) {
   first = 2
   print(first)
   print(" ")
   print(second)
}
void main() {
   int val1 = 5
   int val2 = val1 + 1
   foo(val1, val2)
}
```

Assume that the `print` subprogram prints its output to the screen on a single line. **Provide the output** of the program code if it is assumed that the programming language uses pass-by-name for all parameter passing. Provide only the output as it would be printed to the screen, without any additional text (i.e. additional spaces, quotation marks, or other punctuation). If the program produces an error of any kind, type "Error" without the quotation marks. Note that the second `print` subprogram call outputs a single space.

1 points

QUESTION 21

1. Consider the following program code in a hypothetical programming language:

```
var A = 2

main() {
   var A = 5
   call sub1(sub2)
}

sub1(param) {
   var A = 3
   call param()
}

sub2() {
   print(A)
}
```

Assume the programming language allows subprograms to be passed as parameters, that the `print` subprogram prints its output to the screen on a single line, and that execution starts with the `main` subprogram. **Provide the output** of the program code if deep binding is used to determine the referencing environment of the passed subprogram, but the programming language uses dynamic scoping rules. Provide only the output as it would be printed to the screen, without any additional text (i.e. additional spaces, quotation marks, or other punctuation). If the program produces an error of any kind, type "Error" without the quotation marks.

1 points

QUESTION 22

1. Consider the following program code in C#:

```
public static void doSomething<T>(T p1, T p2) {
   Console.WriteLine(p1 + " " + p2);
}
```

Assume that the `doSomething` method is part of the `Program` class. Also assume that the following program code is provided in the `main` method of the `Program` class:

```
Program.doSomething(5, 2);
Program.doSomething(false, true);
```

Consider the following statements about the use of the `doSomething` method, and **select** the one that is the most correct.

- One version of the `doSomething` method is created at compile time, and is used with `int` and `bool` parameters.
- Two versions of the `doSomething` method are created at compile time. One is used with `int` parameters, while the other is used with `bool` parameters.
- One version of the `doSomething` method is created at runtime, and is used with `Object` parameters.
- Two versions of the `doSomething` method are created at runtime. One is used with `int` parameters, while the other is used with `bool` parameters.

1 points

QUESTION 23

1. Consider the following program code in a hypothetical programming language that supports coroutines:

```
main() {
   resume first
}

couroutine first {
   print("X ")
   resume second
   print("Y ")
}

coroutine second() {
   print("A ")
   resume first
   print("B ")
}
```

Assume that the `print` subprogram prints its string output to the screen on a single line, and that execution starts with the `main` subprogram. **Provide the output** of the program code. Provide only the output as it would be printed to the screen, without any additional text (i.e. additional spaces, quotation marks, or other punctuation). If the program produces an error of any kind, type "Error" without the quotation marks. Note that there is a single space at the end of each of the outputs produced by the `print` subprograms.

1 points

QUESTION 24

1. Abstract data types (ADTs) aim to hide the representations of ADT objects from from the clients that use them. Consider the following possible programming language features in C++, and **select** only the ones that are correct and detract from the effectiveness of ADT representation hiding in C++. Note that incorrectly selected options will be penalised.

- [ ] The existence of `friend` functions and `friend` classes.
- [ ] The fact that C++ supports stand-alone functions that are not part of a class.
- [ ] The fact that C++ requires that classes and member function definitions must appear in the same file.
- [ ] The fact that all class data members must appear in header files.

2 points

QUESTION 25

1. Consider the following statements about abstract data types (ADTs) in Ruby, and **select** the one that is the most correct.

- ( ) Instance and class variable names are more readable in Ruby than they are in Java.
- ( ) Ruby supports multiple constructors, each of which can be implicitly called when `new` is used with the appropriate class name and parameters.
- ( ) It is impossible to implement the functionality of multiple constructors in Ruby.
- ( ) The interface of a Ruby ADT is very reliable.

1 points

QUESTION 26

1. Imagine a hypothetical programming language in which a subclass can inherit a public method from a parent class, and change its access control to private or protected. Consider the following statements about the hypothetical programming language, and **select** the one that is the most correct.

- ○ In the hypothetical programming language, dynamic binding is less efficient.
- ○ In the hypothetical programming language, dynamic binding is impossible.
- ○ In the hypothetical programming language, subclasses are subtypes.
- ○ In the hypothetical programming language, subclasses are not subtypes.

1 points

QUESTION 27

1. Consider the following options, and **select** only the approaches that makes Smalltalk more orthogonal than Java. Note that incorrectly selected options will be penalised.

- ☐ The difference in approach to the exclusivity of objects in Smalltalk and Java.
- ☐ The difference in approach to dynamic and static binding in Smalltalk and Java.
- ☐ The difference in approach to subclasses and subtypes in Smalltalk and Java.
- ☐ The difference in approach to object deallocation in Smalltalk and Java.

2 points

QUESTION 28

1. Consider the following statements about object-oriented programming in C++ and C#, and **select** the one that is the most correct.

- ○ C++ provides worse support for multiple inheritance than C# does.
- ○ C++ provides less writability in terms of nested classes than C# does.
- ○ C++ uses a more orthogonal approach to object-oriented programming than C# does.
- ○ C++ provides a more readable approach to dynamic message binding than C# does.